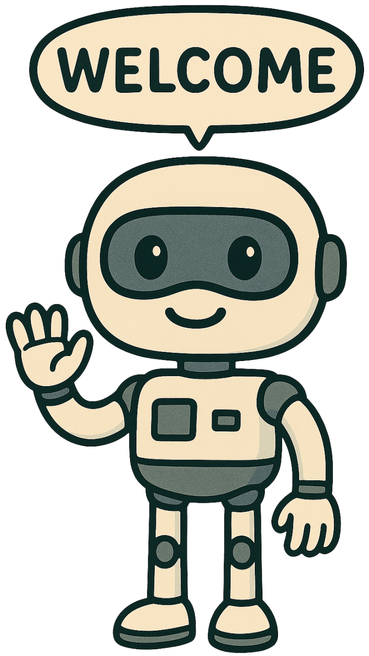
L1: Course Introduction

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca



Who Am I?

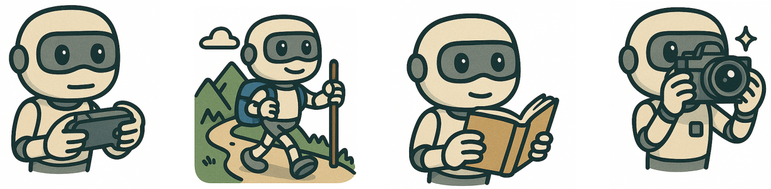
Undergrad from UBC in Engineering Physics.

MSc and PhD from The University of Texas at Austin in Computational Science, Engineering, and Mathematics.

Postdoctoral research at Mila - The Québec AI Institute.

Assistant professor in Electrical and Computer Engineering.
Research: Theory, algorithms, and applications of AI-driven autonomous systems in robotics and engineering.

Hobbies: Reading, photography, hiking, gaming.



Contact: Piazza, or cyrus.neary@ubc.ca.

Who are all of you?

What is Sequential Decision-Making?

Sequential decision making

Add languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

Sequential decision making is a concept in control theory and operations research, which involves making a series of decisions over time to optimize an objective function, such as maximizing cumulative rewards or minimizing costs. In this framework, each decision influences subsequent choices and system outcomes, taking into account the current state, available actions, and the probabilistic nature of state transitions.[1] This process is used for modeling and regulation of dynamic systems, especially under uncertainty, and is commonly addressed using methods like Markov decision processes (MDPs) and dynamic programming.[2]

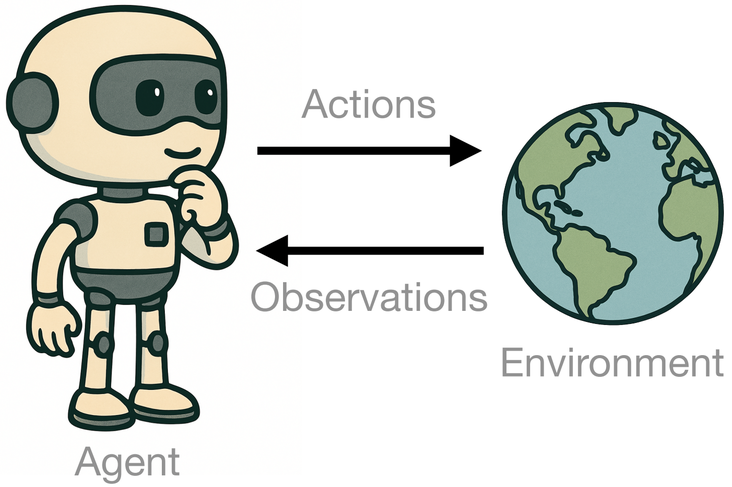
Mathematical modeling
Algorithms and solution techniques
Learning from data
Verifying generated behaviors

Approximate Dynamic Programming

Optimal Control

Reinforcement Learning

Robotics & Autonomous Systems



Game Theory

AI Planning

Economics & Decision Theory

Multiagent Systems

What will we study in this course?

Markov Decision Processes

Markov decision process

23 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

A **Markov decision process** (**MDP**) is a mathematical model for sequential decision making when outcomes are uncertain.[1] It is a type of stochastic decision process[2], and is often solved using the methods of stochastic dynamic programming.

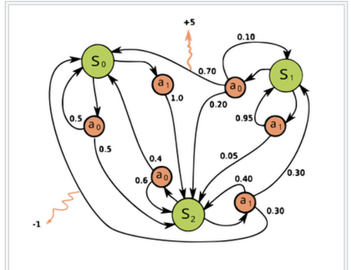
Originating from operations research in the 1950s,[3][4] MDPs have since gained recognition in a variety of fields, including ecology, economics, healthcare, telecommunications and reinforcement learning.[5] Reinforcement learning utilizes the MDP framework to model the interaction between a learning agent and its environment. In this framework, the interaction is characterized by states, actions, and rewards. The MDP framework is designed to provide a simplified representation of key elements of artificial intelligence challenges. This modeling framework incorporates the understanding of cause and effect, the management of uncertainty and nondeterminism, and the pursuit of explicit goals.[5]

The name comes from its connection to Markov chains, a concept developed by the Russian mathematician Andrey Markov. The

Definition [edit]

A Markov decision process is a 4-tuple (S, A, P_a, R_a), where:

- S is a set of states called the *state space*. The state space may be discrete or continuous, like the set of real numbers.
- A is a set of actions called the *action space* (alternatively, A_s is the set of actions available from state s). As for state, this set may be discrete or continuous.
- $P_a(s, s')$ is the probability that action a in state s at time t will lead to state s' at time $t+1$. In general, this probability transition is defined to satisfy $\Pr(s_{t+1} \in S' \mid s_t = s, a_t = a) = \int_{S'} P_a(s, s')ds'$, for every $S' \subseteq S$ measurable. In case the state space is discrete, the integral is intended with respect to the counting measure, so that the latter simplifies as $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$; in case $S \subseteq \mathbb{R}^d$, the integral is usually intended with respect to the Lebesgue measure.
- $R_a(s, s')$ is the immediate reward (or expected immediate reward) received after action a is taken to transition from state s to state s'. The reward is in general a random variable.

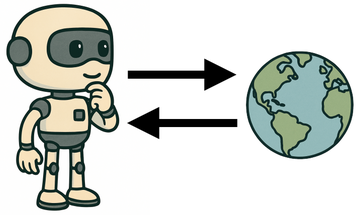


Example of a simple MDP with three states (green circles) and two actions (orange circles), with two rewards (orange arrows)

A policy function π is a (potentially probabilistic) mapping from state space (S) to action space (A).

Mathematical foundations for RL

(and sequential decision-making in general)



Reinforcement Learning

Reinforcement learning

Article Talk

From Wikipedia, the free encyclopedia

For reinforcement learning in psychology, see Reinforcement and Operant conditioning.

In machine learning and optimal control, **reinforcement learning** (**RL**) (*also known as approximate dynamic programming*)[1] is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. Reinforcement learning is one of the three basic machine learning paradigms, alongside supervised learning and unsupervised learning.

While supervised learning and unsupervised learning algorithms respectively attempt to discover patterns in labeled and unlabeled data, reinforcement learning involves training an agent through interactions with its environment. To learn to maximize rewards from these interactions, the agent makes decisions between trying new actions to learn more about the environment (exploration), or using current knowledge of the environment to take the best action (exploitation).[2] The search for the optimal balance between these two strategies is known as the exploration–exploitation dilemma.

The environment is typically stated in the form of a Markov decision process, as many reinforcement learning algorithms use dynamic programming techniques.[3] The main difference between classical dynamic programming methods and reinforcement learning algorithms is that the latter do not assume knowledge of an exact mathematical model of the Markov decision process, and they target large Markov decision processes where exact methods become infeasible.[4]

Why MDPs and RL?

Continuous Control

Flight control

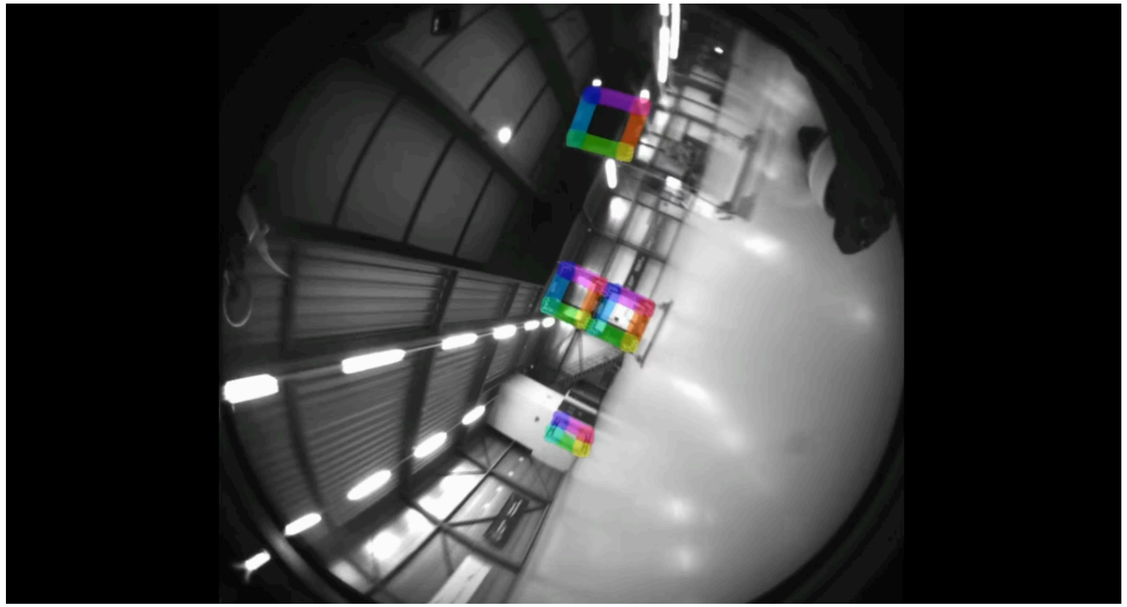
Article

Champion-level drone racing using deep reinforcement learning

https://doi.org/10.1038/s41586-023-06419-4
Received: 5 January 2023
Accepted: 10 July 2023
Published online: 30 August 2023
Open access

Elia Kaufmann[1✉], Leonard Bauersfeld[1], Antonio Loquercio[1], Matthias Müller[2], Vladlen Koltun[3] & Davide Scaramuzza[1]

First-person view (FPV) drone racing is a televised sport in which professional competitors pilot high-speed aircraft through a 3D circuit. Each pilot sees the



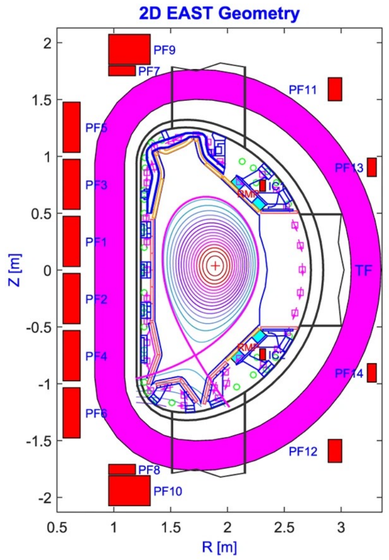
Controlling fusion reactors

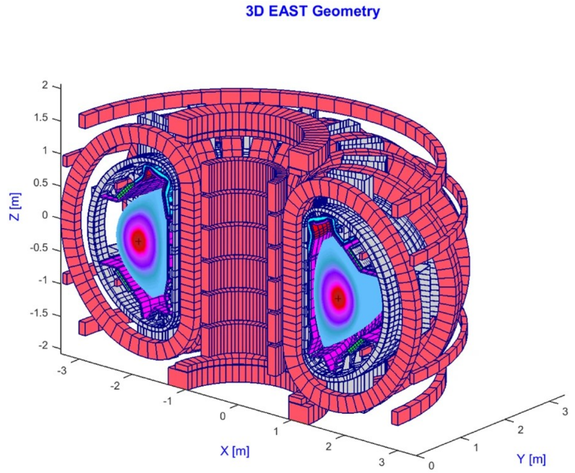
Article

Magnetic control of tokamak plasmas through deep reinforcement learning

https://doi.org/10.1038/s41586-021-04301-9
Received: 14 July 2021
Accepted: 1 December 2021
Published online: 16 February 2022
Open access

Jonas Degrave[1,3], Federico Felici[2,3✉], Jonas Buchli[1,3✉], Michael Neunert[1,3], Brendan Tracey[1,3✉], Francesco Carpanese[1,2,3], Timo Ewalds[1,3], Roland Hafner[1,3], Abbas Abdolmaleki[1], Diego de las Casas[1], Craig Donner[1], Leslie Fritz[1], Cristian Galperti[2], Andrea Huber[1], James Keeling[1], Maria Tsimpoukelli[1], Jackie Kay[1], Antoine Merle[2], Jean-Marc Moret[2], Seb Noury[1], Federico Pesamosca[2], David Pfau[1], Olivier Sauter[2], Cristian Sommariva[2], Stefano Coda[2], Basil Duval[2], Ambrogio Fasoli[2], Pushmeet Kohli[1], Koray Kavukcuoglu[1], Demis Hassabis[1] & Martin Riedmiller[1,3]



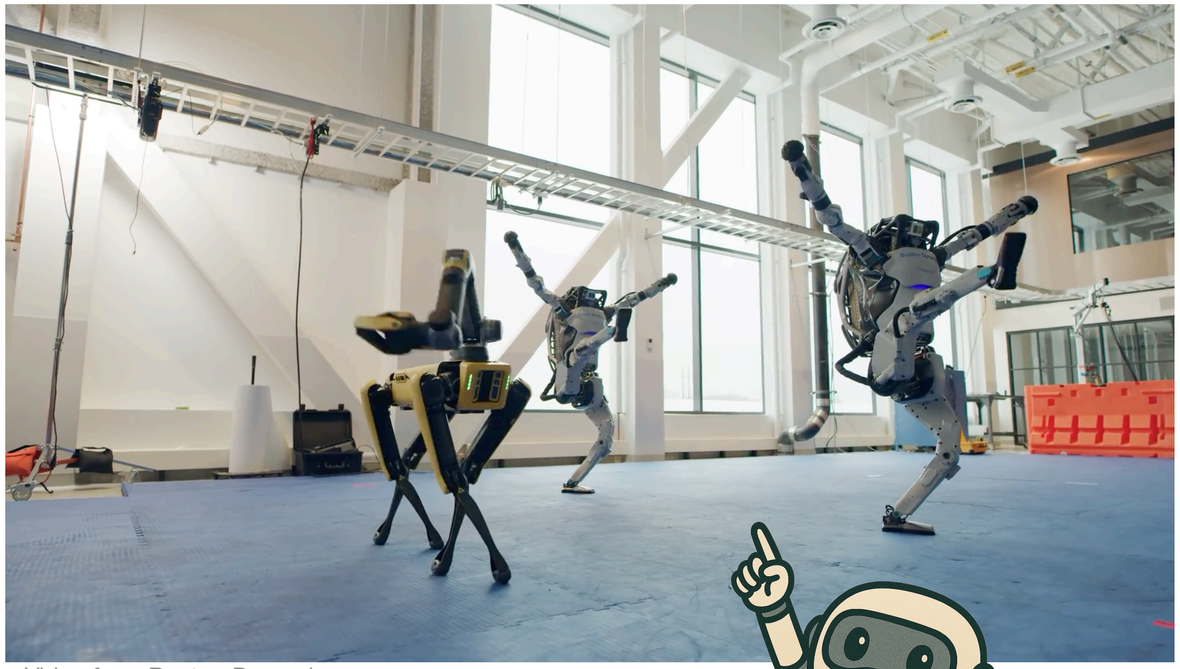


Why MDPs and RL?

Robots…

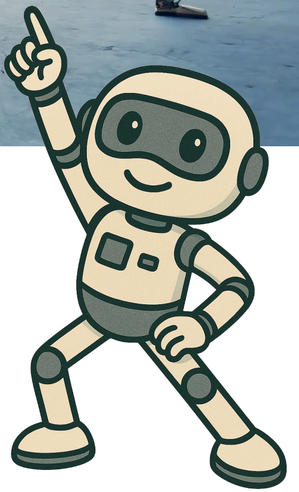
That optimize supply chains…

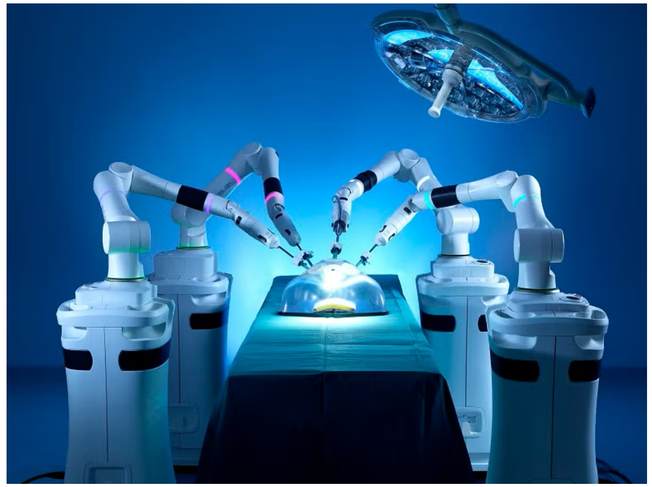
that explore the unknown…



Video from Boston Dynamics

and that dance!

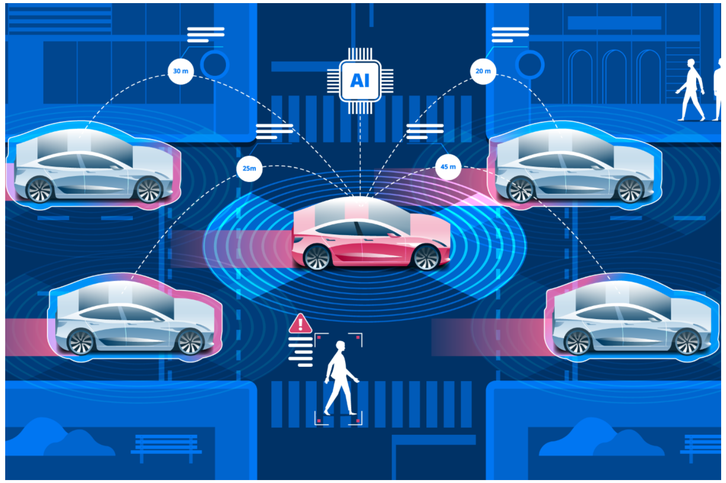




that save lives

Why MDPs and RL?

Autonomous Decision-Making

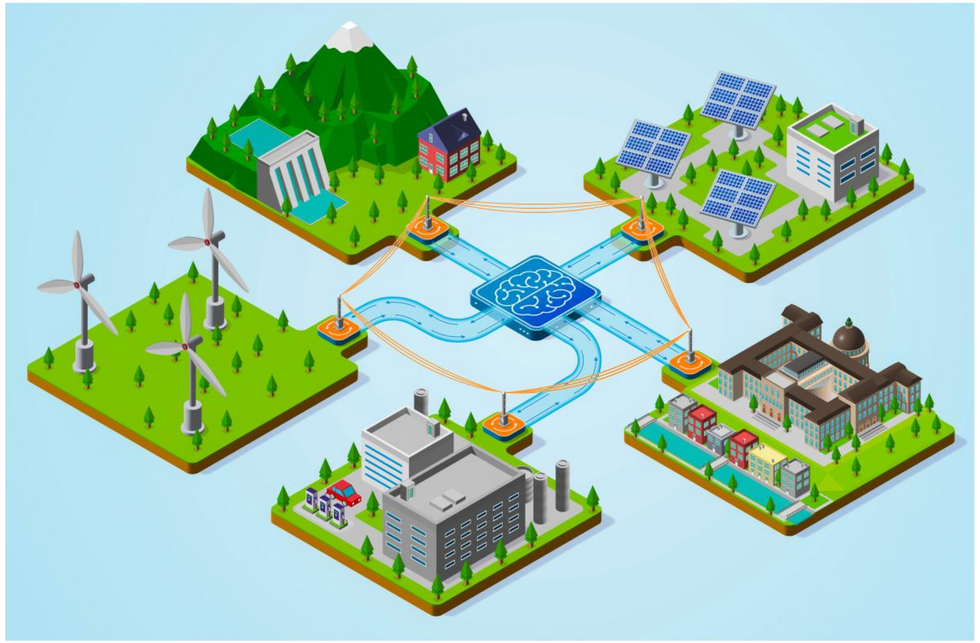


Autonomous vehicles

Automated trading

Why MDPs and RL?

Controlling Networked Systems



Smart power grids

City-wide traffic control

Why MDPs and RL?

Game Playing

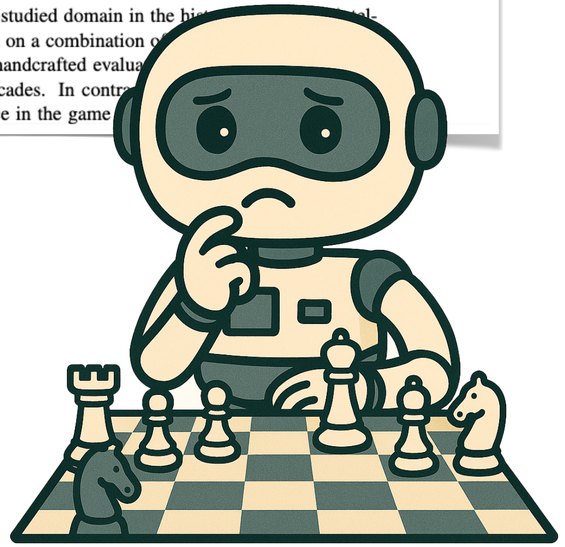
Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm

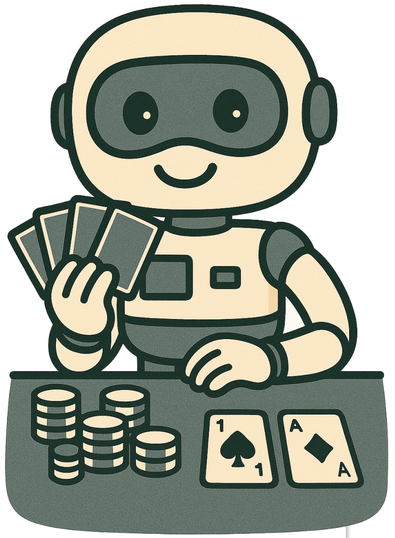
David Silver,[1*] Thomas Hubert,[1*] Julian Schrittwieser,[1*]
Ioannis Antonoglou,[1] Matthew Lai,[1] Arthur Guez,[1] Marc Lanctot,[1]
Laurent Sifre,[1] Dharshan Kumaran,[1] Thore Graepel,[1]
Timothy Lillicrap,[1] Karen Simonyan,[1] Demis Hassabis[1]

[1]DeepMind, 6 Pancras Square, London N1C 4AG.
*These authors contributed equally to this work.

Abstract

The game of chess is the most widely-studied domain in the hi… …l-
ligence. The strongest programs are based on a combination o…
niques, domain-specific adaptations, and handcrafted evalua…
refined by human experts over several decades. In contra…
recently achieved superhuman performance in the game …





RESEARCH ARTICLE

COMPUTER SCIENCE

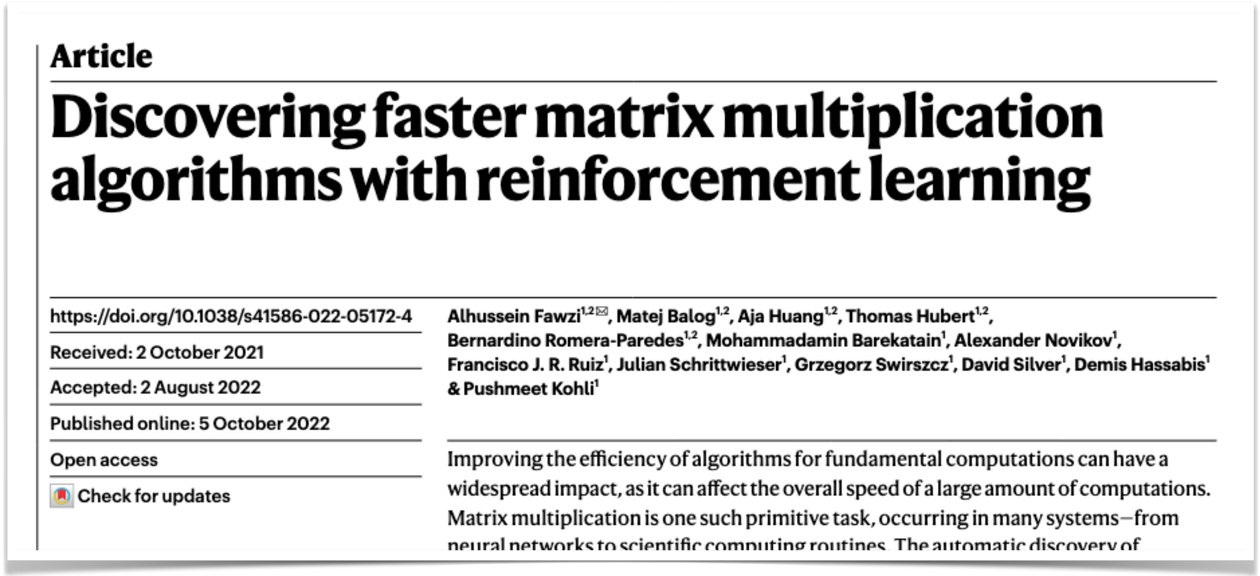
Superhuman AI for multiplayer poker

Noam Brown[1,2*] and Tuomas Sandholm[1,3,4,5*]

In recent years there have been great strides in artificial intelligence (AI), with games often serving as challenge problems, benchmarks, and milestones for progress. Poker has served for decades as such a challenge problem. Past successes in such benchmarks, including poker, have been limited to two-player games. However, poker in particular is traditionally played with more than two players. Multiplayer games present fundamental additional issues beyond those in two-player games, and multiplayer poker is a recognized AI milestone. In this paper we present Pluribus, an AI that we show is stronger than top human professionals in six-player no-limit Texas hold'em poker, the most popular form of poker played by humans.

Why MDPs and RL?

Automating Design and Discovery



Article

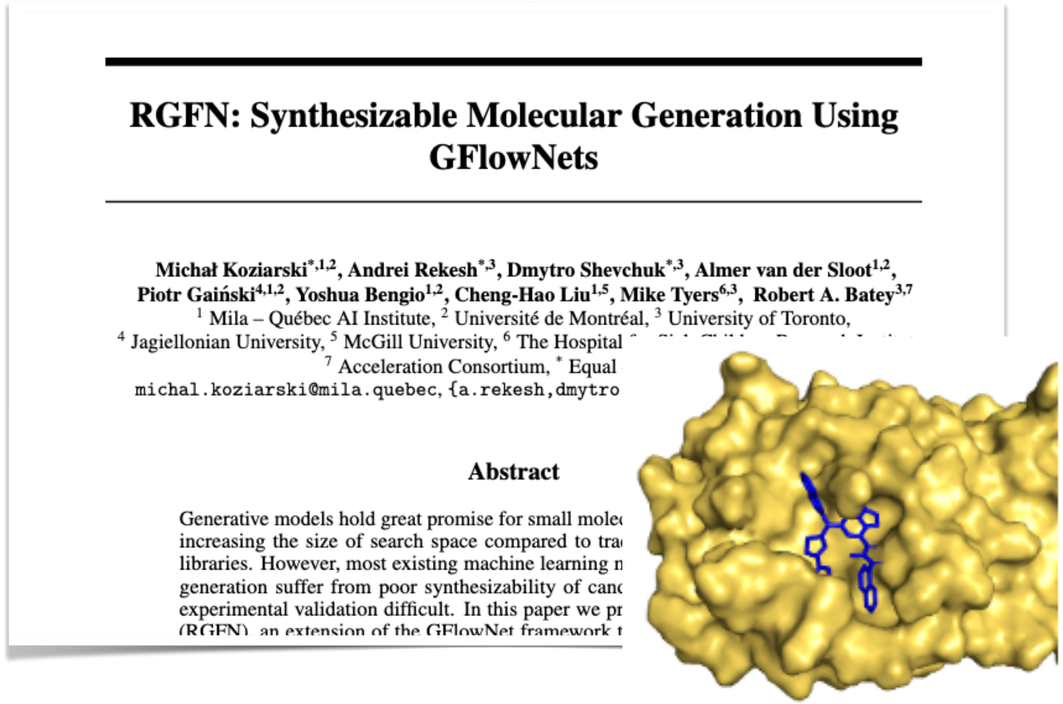
Discovering faster matrix multiplication algorithms with reinforcement learning

https://doi.org/10.1038/s41586-022-05172-4
Received: 2 October 2021
Accepted: 2 August 2022
Published online: 5 October 2022
Open access
Check for updates

Alhussein Fawzi[1,2], Matej Balog[1,2], Aja Huang[1,2], Thomas Hubert[1,2], Bernardino Romera-Paredes[1,2], Mohammadamin Barekatain[1], Alexander Novikov[1], Francisco J. R. Ruiz[1], Julian Schrittwieser[1], Grzegorz Swirszcz[1], David Silver[1], Demis Hassabis[1] & Pushmeet Kohli[1]

Improving the efficiency of algorithms for fundamental computations can have a widespread impact, as it can affect the overall speed of a large amount of computations. Matrix multiplication is one such primitive task, occurring in many systems—from neural networks to scientific computing routines. The automatic discovery of

Searching for efficient numerical algorithms



RGFN: Synthesizable Molecular Generation Using GFlowNets

Michał Koziarski[*,1,2], Andrei Rekesh[*,3], Dmytro Shevchuk[*,3], Almer van der Sloot[1,2], Piotr Gaiński[4,1,2], Yoshua Bengio[1,2], Cheng-Hao Liu[1,5], Mike Tyers[6,3], Robert A. Batey[3,7]
[1] Mila – Québec AI Institute, [2] Université de Montréal, [3] University of Toronto,
[4] Jagiellonian University, [5] McGill University, [6] The Hospital
[7] Acceleration Consortium, [*] Equal
michal.koziarski@mila.quebec, {a.rekesh,dmytro

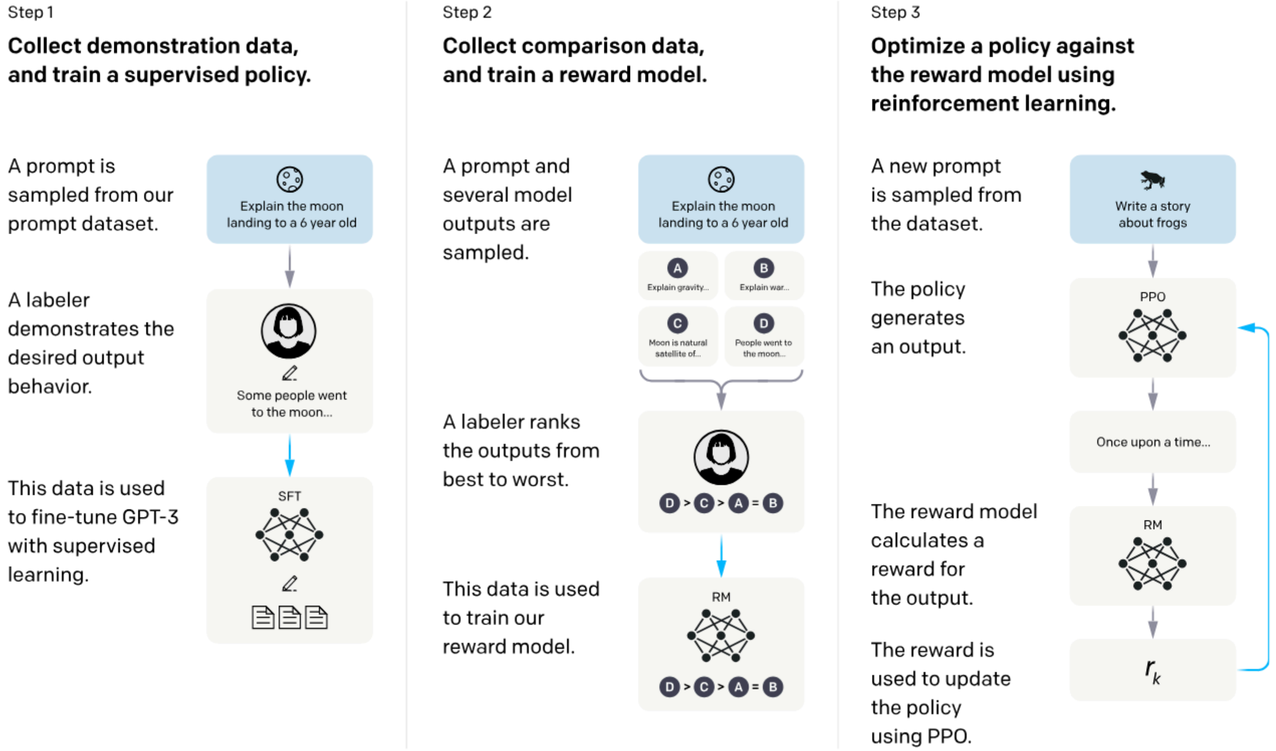
Abstract

Generative models hold great promise for small molec
increasing the size of search space compared to tra
libraries. However, most existing machine learning n
generation suffer from poor synthesizability of canc
experimental validation difficult. In this paper we pr
(RGFN), an extension of the GFlowNet framework t

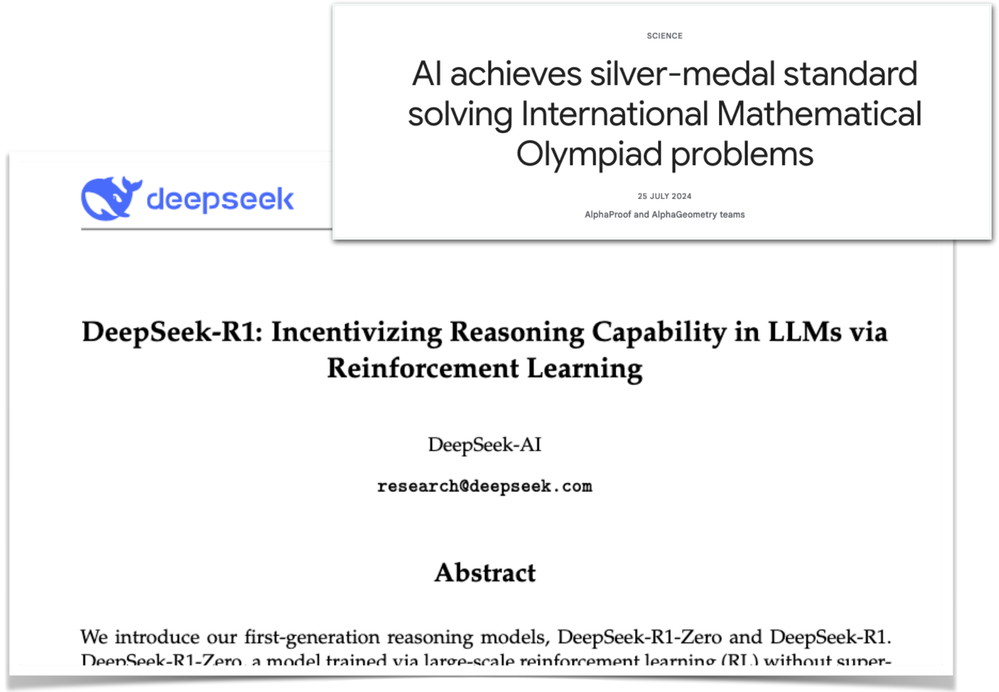
Generating candidate structures for materials and drug design.

Why MDPs and RL?

Language Models and Reasoning



Finetuning language models via RL from Human Feedback.



Reasoning as search via RL in the token space of language models.

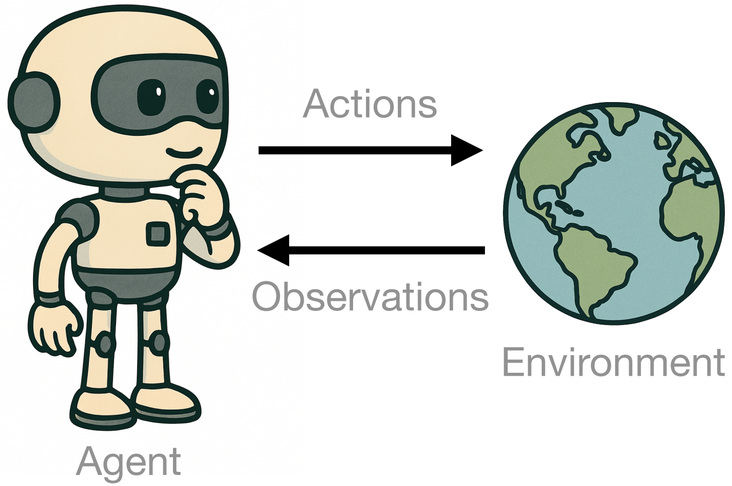
Why MDPs and RL?

Approximate Dynamic Programming

Optimal Control

Reinforcement Learning

Robotics & Autonomous Systems



Game Theory

AI Planning

Economics & Decision Theory

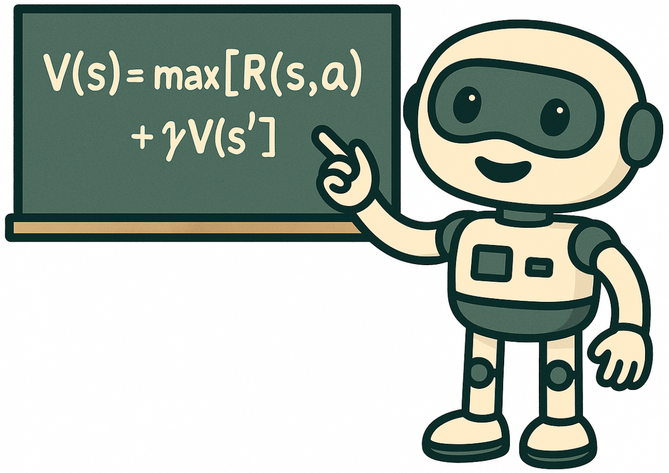
Multiagent Systems

Course Objectives

Formulate real-world decision-making problems mathematically.

Implement and apply dynamic programming and RL algorithms to solve meaningful real-world problems in robotics, control, and autonomous systems.

Critically analyze current research in the field.



Course Outline

Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Part 3

Deep RL algorithms.

- Function approximation
- Deep Q learning
- Challenges with deep RL
- Policy gradient algorithms
- Landmark deep RL algorithms

Part 4

Advanced topics and current research (time permitting).

- Sim-to-real methods
- Intro to hierarchical RL
- Intro to multiagent decision-making

Course Project

The course project will allow students to explore an application or extension of course concepts in depth, and aims to prepare students to complete related independent research projects.

The project will be completed in teams of 3 students and include a short proposal (graded and required for approval by the course instructor), a midterm progress update, and a final written report and in-class presentation.

Project proposals

- Due October 2nd

Project updates

- Due November 6th

Project presentations

- December 2nd and 7th

Project reports

- December 11th

Weekly project surveys

- Every Wednesday at the end of lecture.

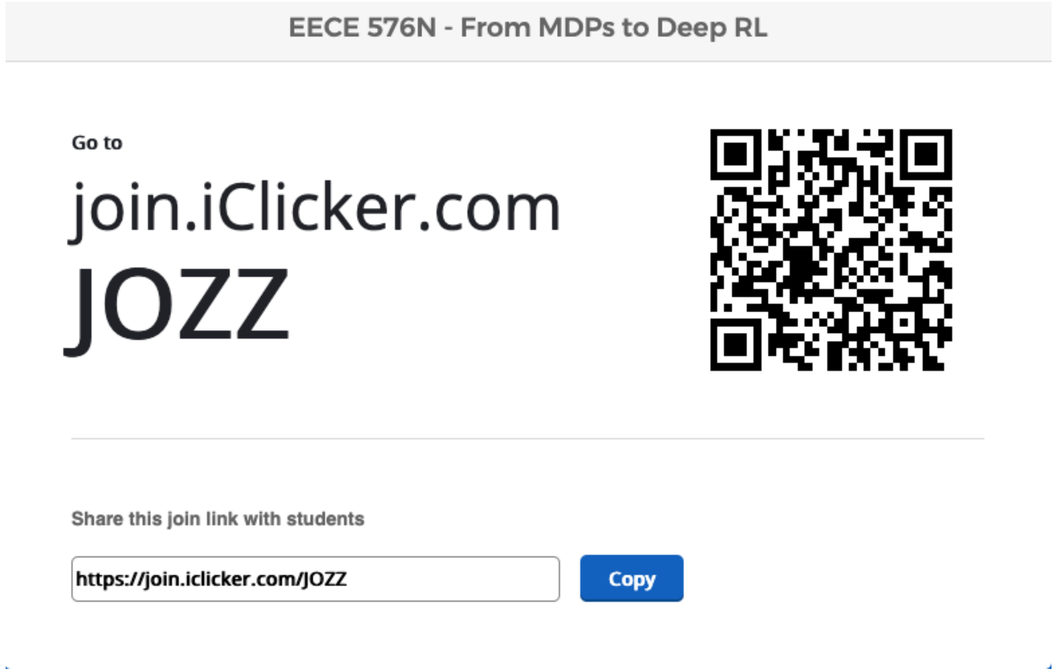
Grading

- 5% - In class participation.
- 20% - Take-home homework and programming assignments.
- 25% - Midterm exam (November 16th in class).
- 50% - Course project.
  - 5% - Project proposal.
  - 5% - Progress update report.
  - 5% - Weekly project update survey (every Wednesday at the end of lecture),
    and individual contribution to project (assessed through responses to the weekly survey).
  - 10% - Final project presentation.
  - 25% - Final project report.

Grading will allow for generative AI models to be used as a research tool. However, all submitted work must be produced by the students and reflective of their own ideas. Submitted assignments or course project materials that are produced primarily by generative AI will receive a score of zero.

Sign-up Links

iClicker



Piazza

https://piazza.com/ubc.ca/winterterm12026/eece576n

Let’s get started!